

Matrix Multiplication & Optimizations

Assignment #2, CSC 746, Fall 2026

Jacob Cordano
Computer Science Department
San Francisco State University
San Francisco, CA, USA
Email: jcordano1@sfsu.edu

Abstract—In this assignment I will implement two matrix multiplication algorithms and analyze their performance against an algorithm from OpeBLAS. Using an AMD EPYC 7763 processor on the NERSC Perlmutter supercomputer, I test matrix sizes ranging from 64 to 2048 to measure MFLOPs over three matrix multiplication algorithms. The applications share identical calculations, with varied levels of performance that differ among each algorithm applied. At their peak performance, the Basic implementation ran at 2568.72 MFLOP/s, the Blocked implementation at 4736.17 MFLOP/s, and the BLAS algorithm ran at 51787.99 MFLOP/s. In each test the findings result in the BLAS algorithm sustaining the highest performance, which can be attributed to the optimizations implemented by OpenBLAS.

I. INTRODUCTION

The goal of this assignment is to understand how the computation of matrix multiplication performs given various optimizations. We want to study how matrix multiplication performs given different applications.

We can evaluate this by performing a baseline with a basic algorithm that multiplies two matrices, a blocked implementation that utilizes various cache sizes for each matrix size, and a highly optimized vendor algorithm from OpenBLAS. Through each implementation, we can evaluate the algorithm's performance through benchmark analysis.

Finally, we can see from our results that the vendor application continues to perform at the highest level whereas my implementations do not closely compare. In contrast, I gain insight and understanding into how cache optimization improves performance over a basic multiplication algorithm.

II. IMPLEMENTATION

A. DGEMM Basic

For the DGEMM Basic algorithm, I will need the ability to multiply two squared matrices and store the

output to memory. The matrices being computed will be of various sizes to allow for experimentation with different problem sizes. The matrices will be stored in a 1D array, and the output will be stored in a 1D array as well. When the function is called, the matrices will be passed in as pointers to the first element of the array. The function will then iterate through the matrices and perform the multiplication and addition to store the output in the output array.

The DGEMM Basic algorithm is a triple nested loop that iterates through the rows and columns of the input matrices. The outer loop iterates through the rows of the first matrix, the middle loop iterates through the columns of the second matrix, and the inner loop iterates through the columns of the first matrix and rows of the second matrix. The inner loop performs the multiplication and addition to store the output in the output array.

For the multiplication of two square matrices, the number of floating point operations (FLOPs) can be calculated as $2 * N * N * N$, where N is the size of the matrix. This is because for each element in the output matrix, I perform N multiplications and $N - 1$ additions, resulting in a total of $2 * N * N * N$ FLOPs for the entire matrix multiplication [1] [3].

```
1 void square_dgemm(int n, double* A, double* B,  
   double* C)  
2 {  
3     for (int i = 0; i < n; i++) {  
4         for (int j = 0; j < n; j++) {  
5             for (int k = 0; k < n; k++) {  
6                 C[n * i + j] += A[n * i + k] * B  
                   [n * k + j];  
7             }  
8         }  
9     }  
10 }
```

Listing 1. Implementation of multiplying two square matrices

B. DGEMM Blocked

The DGEMM Blocked algorithm maintains the same triple nested loop structure as the DGEMM Basic algorithm, but it adds an additional layer of blocking to improve cache performance. The outermost loops iterate through the blocks of the input matrices, while the innermost loops iterate through the elements within each block. This allows for better cache utilization and leads to improved performance.

Number of blocks is calculated as $N/\text{block_size}$, where N is the size of the matrix and block_size is the size of the blocks. The number of blocks is used to determine how many times to iterate through the outermost loops [2].

```
2 dgemm_blocked(n, block_size, A, B, C)
3   num_blocks = n / block_size

5   for i from 0 to num_blocks - 1:
6       for j from 0 to num_blocks - 1:
7           C_cache = C[i, j]

9           for k from 0 to num_blocks - 1:
10              A_cache = A[i, k]
11              B_cache = B[k, j]

13              C_cache = C_cache + (A_cache *
14                  B_cache)

15              C[i, j] = C_cache
```

Listing 2. Pseudo code representation of `dgemm_blocked()` function.

C. DGEMM BLAS

The DGEMM BLAS algorithm utilizes the Basic Linear Algebra Subprograms (BLAS) library from OpenBLAS to perform matrix multiplication. The BLAS library is a highly optimized library for performing basic linear algebra operations, including matrix multiplication [5]. By using the BLAS library, I can take advantage of its optimized implementations and achieve better performance compared to my own implementations. The DGEMM BLAS algorithm calls the `cblas_dgemm()` function from the BLAS library, which performs the matrix multiplication and stores the output in the output array.

D. Timer

For the timer, I used the `std::chrono` library from the C++ standard library to measure the elapsed time of the matrix multiplication. The timer is started before the matrix multiplication is performed and stopped after the multiplication is complete. The elapsed time is then

calculated by subtracting the start time from the end time and converting the result to seconds.

```
1 Start timer
3 If blocked implementation is enabled:
4     Run blocked DGEMM using N and block size
5 Else:
6     Run basic DGEMM using N
8 Stop timer
10 Compute elapsed time in seconds
12 FLOPs = 2 * N * N * N
14 MFLOP/s = FLOPs / (elapsed time * 1,000,000)
```

Listing 3. Pseudocode for timing the matrix multiplication benchmark.

III. EVALUATION

The evaluation of the DGEMM Basic, DGEMM Blocked, and DGEMM BLAS algorithms was conducted on a high-performance computing platform to assess their performance across different matrix multiplication routines. The outcome of this is presented in terms of MFLOP/s, where I compare the performance of each algorithm as the size of the input matrices increases.

A. Computational platform and Software Environment

This evaluation was performed on the NERSC Perlmutter supercomputer using two AMD EPYC 7763 processors and a base clock speed of 2.45 GHz. The system is equipped with 512 GB of DDR4 memory per node, an L1 cache of 32KB, an L2 cache of 512KB, an L3 cache of 32MB, providing ample resources for large matrix multiplications [4]. The software environment utilizes GNU C++14 SUSE Linux, CMake 3.28, the OpenBLAS library for the DGEMM BLAS implementation, and the C++ standard library for timing and benchmarking.

B. Methodology

To test the performance of the matrix multiplication algorithms, I have incorporated a set of test sizes with matrices that are of $N = \text{size } 64, 128, 256, 512, 1024, \text{ and } 2048$. With those test sizes, I ran a program that captures FLOPs at using the $2 * N * N * N$ formula. Finally, the MFLOPS/s is calculated by dividing the FLOPs by the elapsed time in seconds and multiplying by 1,000,000 to convert to millions of FLOPs per second. For the BLAS implementation, I disregarded the first of two $N = 64$ tests to account for the cache warm up. With this

MFLOP/s data, I was then able to record and graph the results for the analysis below.

C. Comparison of Basic MM and CBLAS

When comparing the performance of the DGEMM Basic and DGEMM BLAS algorithms, I found that the DGEMM BLAS implementation consistently outperformed the DGEMM Basic implementation across all tested matrix sizes. As the problem sizes increased, I saw performance decrease for the Basic implementation. This is likely due to the lack of optimizations in the Basic algorithm has compared to the BLAS library. I will also note the highest performance in the Basic implementation is likely due to the higher cache hit rate with the smaller problem size.

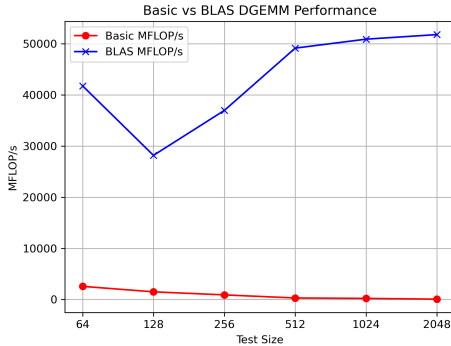


Fig. 1. Here we can see the BLAS implementation always outperforms the Basic implementation.

D. Blocked MM with Copy Optimization (BMMCO), and CBLAS

In the comparison between DGEMM Blocked and DGEMM BLAS, the BLAS implementation outperformed the blocked implementation for every tested matrix size. Although blocking improved performance relative to the basic implementation, it did not match the performance of BLAS. For the Blocked implementation, I found the highest performance was achieved with block sizes greater than 2, with the best performance being achieved with a block size of 16. The performance of the Blocked implementation decreased as the block size increased beyond 16, likely due to the increased overhead of managing larger blocks and the reduced cache efficiency.

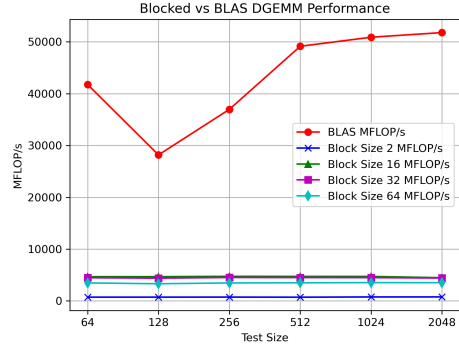


Fig. 2. Similar to the Basic implementation, the Blocked implementation underperforms against the BLAS algorithm.

E. Overall findings and discussion

Overall, when comparing the performance of the Basic, Blocked, and BLAS implementations, I found the BLAS implementation consistently outperforming across both the Basic and Blocked implementations. While the Basic and Blocked implementations fell short of the performance of BLAS, they still provided valuable insights into the impact of performance optimizations on matrix multiplication. When comparing the Basic and Blocked implementations, I found the strongest similarities lie between the small problem size of the Basic and Blocked implementation at $N = 64$. This can be explained as both algorithms functioning identically in the way they process this data size. Outside of those test sizes, the Blocked implementation remained the stronger performer over Basic.

Because the cache line of the L1 Cache is 32KB (or 32,768 bytes), we can see why the top performers are the 16 and 32 block sizes. We can calculate this by multiplying the number of matrices, by the size of the blocks, by the size of the number of kilobytes per 64 byte cache line as so: $3 * (B * B) * .008$. For the block of 16, that is 6.14KB and for a block of 32, that is 24.58KB. When we step up to a block size of 64, the cache required is 98.3KB, which far exceeds the L1 cache size.

On the other side of the spectrum, the cache required for the block size of 2 is .1KB. While this does fit into the L1 cache, the unnecessary work created by processing such small blocks prohibits performance.

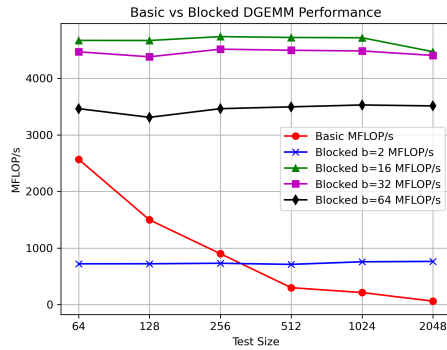


Fig. 3. Here is a closer look at the comparison between the Basic and Blocked implementations, with the Blocked algorithm consistently outperforming.

IV. AI USAGE

In order to complete this assignment AI was used in the following ways:

- To configure the included Dockerfile so that I may easily run the homework file without installing various packages locally to my machine.
- After running the benchmarks, I would ask an LLM to combine data into one CSV for plotting.
- For LaTeX debugging, formatting, and learning how to use citations.
- For understanding lecture notes, which is how I derived the equation for calculating the amount of cache used for each test size ($3 * (B * B) * .008$).
- To properly plot my Blocked VS Basic graph.
- To convert my code into pseudocode.

REFERENCES

- [1] Bethel, E. Wes. Csc 746 f26 lecture 05 – cp#1 review, optimizing cache use, cp#2, Sep 2026.
- [2] Bethel, E. Wes. Csc 746 f26 lecture 06 – optimizing use of cache, Sep 2026.
- [3] mCoding, Nov 2021.
- [4] Nersc. Perlmutter architecture.
- [5] OpenBLAS Developers. Openblas faq, Jul 2026.